



AmazonAICloud: proactive resource allocation using amazon chronos based time series model for sustainable cloud computing

Han Wang¹ · Kevin Joji Mathews¹ · Muhammed Golec¹ · Sukhpal Singh Gill¹ · Steve Uhlig¹

Received: 29 August 2024 / Accepted: 3 February 2025 / Published online: 18 February 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Austria, part of Springer Nature 2025

Abstract

Efficient cloud resource allocation is critical for optimising cloud computing services in terms of minimising operational costs and improving energy efficiency. The increasing demand for cloud services results in considerable energy consumption. In order to improve cloud sustainability and reduce energy consumption, the latest Artificial intelligence (AI) techniques, such as deep learning, can be applied to accurately predict resource utilisation, thereby optimising cloud performance. This paper introduces a novel framework, AmazonAICloud, that integrates Amazon Chronos, a time-series forecasting model, to forecast CPU utilisation and enhance resource allocation in cloud computing environments. The AmazonAICloud framework utilises a real-world cloud dataset that contains 1750 VM traces from BitBrains. We implement the Chronos model on Amazon web services (AWS) SageMaker and evaluate its performance against the baseline models like Autoregressive integrated moving average (ARIMA), Long Short-term memory (LSTM), and DeepAR using various machine learning-based evaluation metrics. Finally, the experimental results show that the Amazon Chronos model outperforms the baseline models in terms of CPU usage prediction, allowing for effective resource provisioning to reduce operational costs and energy consumption.

Keywords Cloud computing · Machine learning · Sustainable cloud computing · Proactive resource allocation · Time series prediction

Mathematics subject classification 68M20 · 68M14 · 68T05

1 Introduction

Over the years, cloud computing has emerged as a foundational computing paradigm across various industries and revolutionised how organisations and individuals access and manage data by offering adaptive and virtually infinite resources via the Internet [1, 2]. Cloud services support the significant advancements in many information technologies such as Artificial Intelligence (AI) and Internet of Things (IoT) [3–5]. Over the recent years, the rapid development of AI, which requires tremendous computing resources supported by cloud services, has had a significant impact on modern technology, facilitating breakthroughs in data-driven decision-making, intelligent automation, and predictive analytics [6–8]. Its influence extends across diverse sectors, also opening new possibilities for optimising cloud infrastructures [9]. Recent developments highlight the orchestration between cloud technology and AI, which has the potential to improve cloud services, reduce operational costs, and enhance service accessibility [10–12].

1.1 Background and recent advancements

Since the ubiquity of cloud computing services over the past decade, cloud resource management strategies have evolved significantly [13]. These advancements are evident in the scale of cloud environments, including the variety of cloud service types and the ample availability of cloud solutions, as well as the increasing complexity of user demands [14, 15]. Cloud optimisation solutions are moving from heuristic- and rule-based approaches toward more sophisticated and AI-driven techniques [10]. Early works focused on integrated and autonomic scaling solutions, to avoid overly simplistic rules for scaling [16]. Later research began emphasising the interplay between heterogeneous data centre infrastructures and dynamic user demands, also highlighting challenges related to Quality of Service (QoS) and heuristic optimisation, such as the unpredictability of cloud data centres [17]. More recently, attention has shifted from conventional optimisation techniques to Machine Learning (ML) and deep learning-based methods for proactive resources allocation, to improve resource efficiency and accommodate fluctuating workloads across cloud and fog environments [18–20].

1.2 Economic and environmental insights

Although cloud computing services have contributed significant transformative value to the society, their environmental impact cannot be neglected [21, 22]. The rapid growth and adoption of AI models have greatly accelerated the demand for computing resources, much of it is accommodated by large distributed cloud data centre [23]. These cloud data centres are primary drivers of energy consumption in many areas, as operating data centres require a substantial amount of electricity, leading to significant carbon footprints, they also consume large amounts of water for its cooling system [24–26]. According to the International Energy Agency

(IEA) that the all cloud data centres in the United States are estimated to account for approximately 6% of the national total electricity consumption [26]. These environmental impact of cloud services has drawn attention to the need for practical solutions to mitigate its impact [27, 28].

1.3 Motivation and contributions

Despite the advancements enabled by ML techniques, resource management remains a persistent challenge in cloud computing [12, 29–31]. Inefficient resource distribution continues to cause two significant issues: over-provisioning and under-provisioning. Under-provisioning compromises application performance and QoS; over-provisioning poses a more critical concern as it results in wasted resource and energy consumption [32]. Figure 1 illustrates a typical example of resource over-provisioning, where excessive resources are provisioned but remain underutilised. Traditional resource allocation solutions such as auto-scaling and load balancing in cloud services usually rely on threshold-based rules by monitoring the CPU utilisation [33]. While these approaches can be effective in the short term, they may not be able to anticipate sudden future demand spikes and thus lead to underperformance or wasted resources [34]. Therefore, incorporating a more dynamic scaling approach is crucial for optimising cloud resource allocation. Recent studies have demonstrated that deep learning and genetic algorithms can be employed to predict future CPU utilisation more accurately, enabling proactive resource provisioning that can mitigate over-provisioning and under-provisioning [35].

However, existing research primarily relies on models such as Long Short-Term Memory (LSTM), Recurrent Neural Network (RNN), Autoregressive Integrated Moving Average (ARIMA) [36]. While these models are effective for relatively simple prediction tasks where the time sequence patterns are highly correlated [37], but they often struggle to handle the rapidly changing and non-stationary time series data in real-world cloud environments [38]. Moreover, recent study has highlighted the need for practical solutions that can integrate into real-world systems without the necessity of significant overhead for model training and deployment [37].

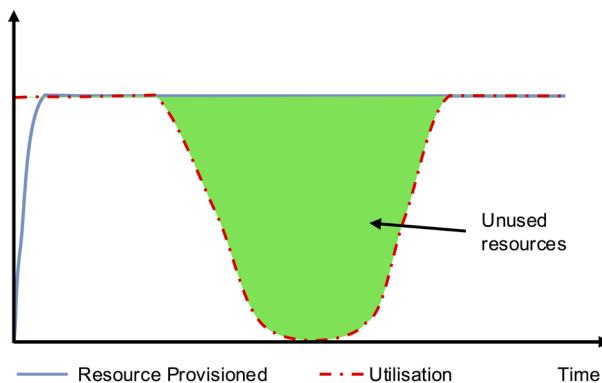


Fig. 1 Example of resource wastage due to over-provisioning

More recent work frameworks such as Amazon Chronos, that have been specifically designed to deal with complex time-series prediction environments, are more suited for non-stationary datasets, which are typical in real-world scenarios [39]. Chronos is a pretrained model and available in multiple version sizes, making it particularly suitable for diverse datasets and more efficient for practical deployment.

The primary objective of this paper is to optimise cloud resource prediction to improve resource allocation strategies, thereby promoting more efficient and sustainable cloud computing services. To achieve this, we propose AmazonAICloud, a framework designed to forecast future CPU usage for optimised cloud resource management. This framework incorporates a time-series forecasting model integrated with Amazon Chronos. By enabling more precise resource provisioning, it can contribute to reducing resource wasted, which is a critical factor in improving sustainability of cloud computing, and minimising cost and CO₂ emissions. The main contributions of this paper are:

- We propose the AmazonAICloud framework, which uses the Amazon Chronos model to predict cloud workload demand and optimise cloud resource allocation for sustainable computing.
- We examine and highlight the challenges in cloud time-series data, particularly issues related to the stationarity of the dataset and potential limitations of existing models in handling these complexities.
- We employ AWS SageMaker to implement the Chronos model and predict future CPU utilisation for efficient resource allocation.
- We evaluate the performance of the Amazon Chronos model against baselines including ARIMA, LSTM, and DeepAR using the GWA-T-12 cloud dataset from Bitbrains in terms of mean absolute error (MAE), mean squared error (MSE), and mean absolute percentage error (MAPE).
- We demonstrate that the Amazon Chronos model in the AmazonAICloud framework outperforms the baseline models in terms of prediction performance, while also offering lower overall overhead and ease of model development.

1.4 Paper organisation

The rest of the paper is organised as follows: Section 2 provides an overview of the related works in cloud resource management. Section 3 discusses the proposed framework and methodology. Section 4 presents a performance evaluation of the proposed model with baseline models and discusses experimental results. Section 5 concludes the paper and highlights the potential future directions.

Table 1 Comparison of the Proposed AmazonAICloud Framework with Existing Works

Work	Type	Feature	Prediction model	Strategy	Environment	Metrics	Dataset
Calheiros et al. [34]	Container	Request Rate	ARIMA	Reactive	CloudSim	Average Service Time, Prediction accuracy	Wikimedia Foundation
Tang et al. [40]	Container	CPU	Bi-LSTM	Proactive	Kubernetes Cluster	Prediction Accuracy, RMSE	Traced from data centre with 500 containers
Yan et al. [41]	Container	CPU, Memory	Bi-LSTM	Hybrid	Kubernetes Cluster	SLA Conflicts, CPU Utilisation	Alibaba Cluster Trace
Christofidi et al. [37]	VM	CPU, Memory, Network work	Persistent Forecast, LSTM	Proactive	Custom Environment	MAE, Average Relative Delta	Alibaba Cluster Trace, Azure VM Traces, Google Cluster Trace, Bitbrains
Ali et al. [42]	VM	CPU, Memory, Network work	DENN	Proactive	Custom Environment	RMSE, R-squared, Mean Bias Deviation, Uncertainty Coefficient	Google Cluster Trace, Alibaba Cluster Trace
Velu et al. [43]	VM	CPU	DeepAR	Proactive	Google Colab	Prediction Accuracy, MAE, MSE, MAPE, CPU Usage, Execution Time, Energy cost	Bitbrains
Proposed model (AmazonAICloud)	VM	CPU	Amazon Chronos	Proactive	AWS Sagemaker	Prediction Accuracy, MAE, MSE, MAPE	Bitbrains

2 Related work

This section examines relevant works in the area of cloud resource optimisation. Table 1 compares our proposed AmazonAICloud framework with previous works based on key parameters. Early work by Calheiros et al. [34], employed a reactive ARIMA-based prediction approach on CloudSim to predict the number of requests, achieving an average accuracy of up to 91%. While these reactive approaches demonstrate impressive accuracies, the primary limitation is that they can only respond after observing fluctuations in the utilisation, which can result in degraded QoS [34].

In contrast, proactive allocation approaches predict workloads and reallocate resources before changes occur. Tang et al. [40] proposed a container-based workload prediction model that utilises a Bidirectional Long Short-Term Memory (Bi-LSTM), achieving over 50% prediction accuracy and a significantly lower Root Mean Square Error (RMSE) of 0.1 compared to traditional ARIMA and LSTM models. Similarly, Yan et al. [41] introduced an adaptive horizontal scaling system on Kubernetes. Their Bi-LSTM-based model accurately predicted the cloud workload and improved the resource utilisation by 20%.

Christofidi et al. [37] investigated whether complex ML models such as LSTM, are strictly required for predicting resource usage in cloud environments. They conducted extensive experiments using cloud dataset such as Alibaba, Azure and Google, which show that some of these workloads exhibit strong temporal correlations. These characteristics of dataset enable the use of simple “persistent forecast”, which can achieve comparable prediction accuracy and does not necessarily require complex ML models in case of stable cloud workload [37]. While they highlight the importance of evaluating the stability of datasets before adopting ML models such as LSTM and emphasise the overhead associated with ML solutions. This underscores the need for a more sustainable and efficient time-series prediction framework for cloud resource management.

Ali et al. [42] propose a cluster-based differential evolution neural network (CDENN) for proactive workload prediction in cloud data centres. Their approach utilises a deep neural network with an evolutionary algorithm that has a mutation strategy, its automatic process of feature tuning can reduce the complexity of development. Their experiments using Google and Alibaba real-world traces demonstrate accurate predictions with an error rate of 0.0002. While this study does not consider the sustainability issues of cloud services. Velu et al. [43] developed an AI-based resource predication approach, which employed the DeepAR model to forecast CPU utilisation for a given workload. The proposed approach was able to reduce the percentage of unused CPU to 32.35% as compared to traditional approaches.

2.1 Critical analysis

The studies in Table 1 have advanced the cloud resource management by improving efficiency and sustainability. However, most of these works operate under assumptions or data conditions that are stationary. Real-world cloud workloads often exhibit

non-stationary characteristics where sudden spikes can affect predictive accuracy, depending on application scenarios. In such cases, it could lead to compromising models' performance, resulting in over-provisioning or under-provisioning if inaccurate predictions are made. While Velu et al. [43] have improved these issues through a proactive DeepAR model for forecasting CPU usage. There remains a need for lightweight and robust predictive solutions that are adaptable to rapidly changing and non-stationary cloud scenarios as highlighted by [37].

3 Methodology

This section describes the architecture of the AmazonAICloud framework and its case study, along with the cloud dataset, data preprocessing, and model training process.

3.1 Cloud dataset

In this study, we employ the Bitbrains cloud VM trace dataset (GWA-T-12), containing the performance metrics from 1750 VMs [44]. In the dataset, each of the VMs is recorded with a number of metrics including CPU usage, CPU provisioned, CPU cores, memory usage, and memory provisioned over a time span from 2013-07-01 to 2013-10-01. Figure 2 presents the CPU capacity provision and actual CPU utilisation. There is a noticeable gap between the provisioned CPU and actual utilisation, suggesting an inefficient allocation of cloud resources [45].

Previous research [43] evaluated the stationarity of Bitbrains time series data using only the Augmented Dickey-Fuller (ADF) test; however, relying on a single test may not be sufficient to conclusively determine stationarity. To address this concern, we additionally applied the Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test to verify stationarity of the dataset. Unlike the ADF test, the KPSS test assesses stationarity around a deterministic trend [46]. In the KPSS test, the null hypothesis

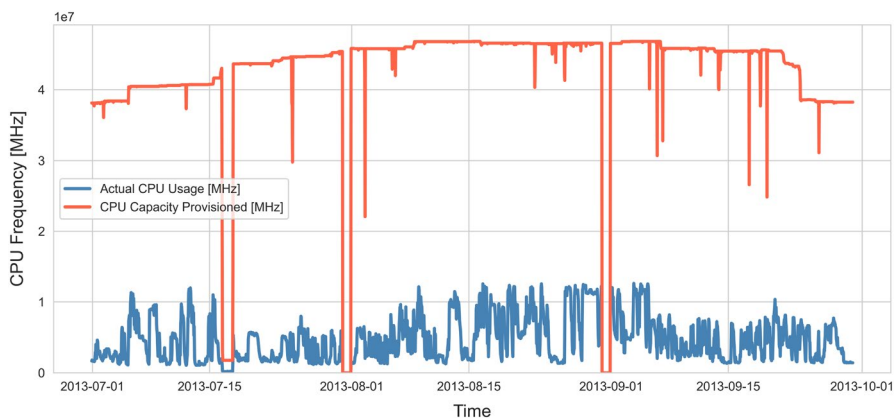


Fig. 2 CPU usage vs CPU provisioned over a 3-Month period

Table 2 Results of the KPSS Test on the Cloud Dataset

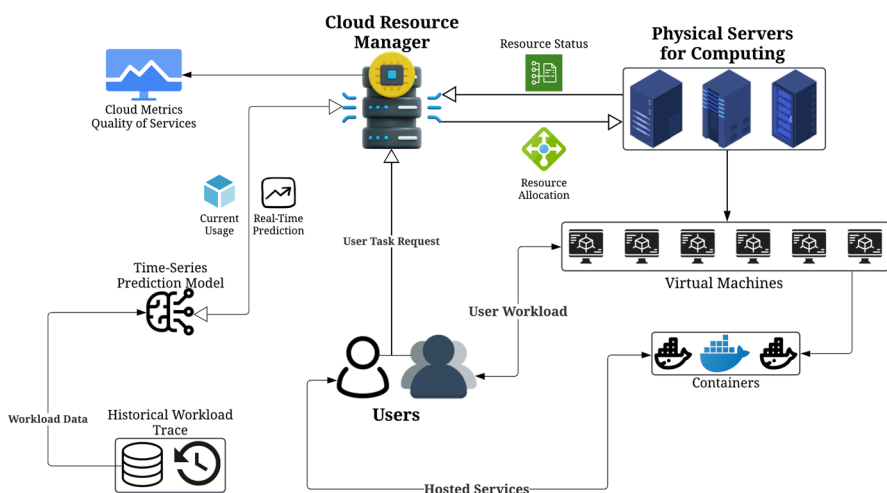
Measure	Value
KPSS statistic	0.432
p-value	0.01
Lags used	27
Critical value (1%)	0.216
Critical value (2.5%)	0.176
Critical value (5%)	0.146
Critical value (10%)	0.119

states that the time series is trend-stationary. Table 2 shows the results of the KPSS test on the cloud dataset.

Our results show that the KPSS statistic of 0.432 exceeds the critical values at 1%, 2.5%, 5%, and 10% significance levels, while its corresponding p -value is below 0.05, indicating that the time series is likely non-stationary. Considering the high possibility of non-stationarity in real-world datasets, relying on conventional approaches that assume stationarity can lead to inaccurate forecasting [47].

3.2 AmazonAICloud framework

In this section, we discuss the AmazonAICloud framework using a real-world case study of its e-commerce platform, which experiences fluctuating demand, to elucidate its various components and demonstrate its step-by-step application in a practical context. Figure 3 shows the architecture of the proposed AmazonAICloud framework in cloud computing environments.

**Fig. 3** Architecture of AmazonAICloud Framework

3.2.1 Real-world case study of E-commerce platform with fluctuating demand

We examine an online shopping platform that experiences unpredictable fluctuations during promotions or seasonal sales [48]. For example, one of the brands on the platform might go viral on social media, leading to a surge in requests for products, browsing and orders. The platform operates on a cloud provider's infrastructure to scale resources dynamically and maintain service quality during such demand spikes. The shopping platform operates in a cloud environment within the following components as shown in Fig. 3:

- **Platform as the User:** The shopping platform interacts with the cloud infrastructure to request and utilise resources for hosted services.
- **Cloud Resource Manager:** The central component of the framework that receives workload traces from the shopping platform, monitors physical servers and exchanges resource status of servers at the same time. It also interacts with the trained time-series prediction model to forecast upcoming resource demands and make optimal resource allocation decisions accordingly.
- **Time-Series Prediction Model (Chronos):** The model trained on historical cloud workload traces makes predictions of short-term workload behaviour based on the current resource metrics and sends the outcome to the resource manager.
- **Physical Servers for Computing:** The underlying hardware that hosts VMs or container services can be provisioned or de-provisioned based on workload demands, as determined by the resource manager.

3.2.2 Amazon chronos model

Figure 4 shows the sequence diagram of Chronos model. Chronos is a pretrained, transformer-based model developed by researchers from Amazon AI Labs [39]. Chronos model is available in five variants (tiny, mini, small, base, and large), which differ in the vocabulary size to suit various computational needs [39]. The tiny and mini models can run on standard CPUs, making them accessible and efficient for many applications. In our study, we utilise the Chronos tiny model as it requires minimal computing resources, allowing us to make predictions using smaller CPUs without the need for powerful hardware like GPUs. The model is configured to generate 30 samples with a prediction length of 30 time steps.

The Chronos model processes time-series data by first scaling it using mean scaling which normalises each data point based on the mean of the absolute values from the initial time steps [39]. The scaled data is then quantized into discrete tokens using a uniform binning technique. Two special tokens, padding token (PAD) and end of sequence token (EOS) are added to manage sequences of different lengths, with PAD token padding shorter sequences to a fixed length and EOS token marking the end of a sequence. During training, Chronos learns from these tokens using cross-entropy loss. For making inferences, the model generates tokens one step at a time autoregressively and then converts these tokens back into numerical values by applying an inverse scaling transformation to produce the forecasted values [39].

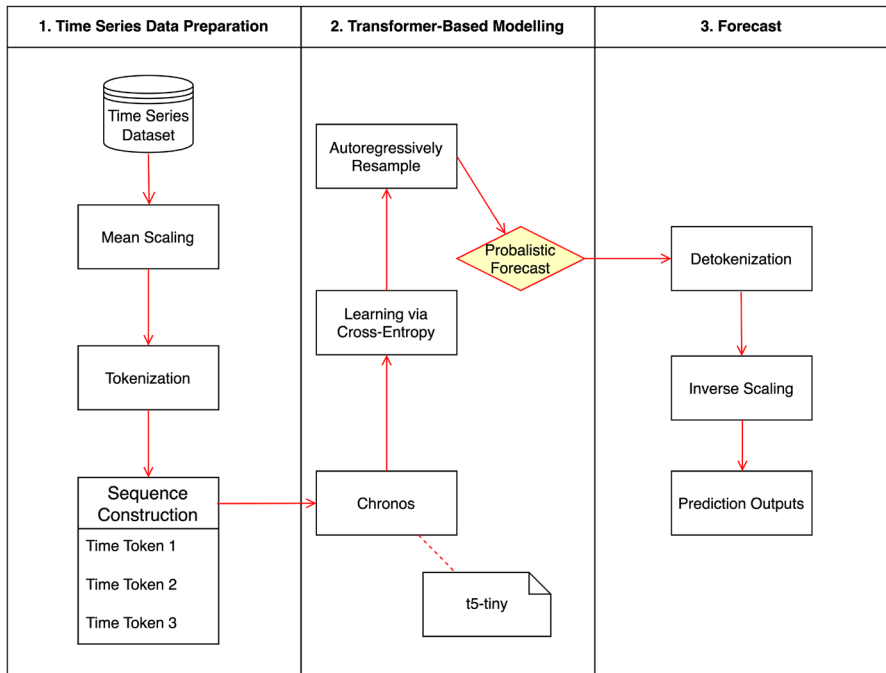


Fig. 4 Sequence Diagram of Amazon Chronos Model

3.2.3 Model training

The training process uses the model parameter (θ) to minimise the cross entropy between the quantized true values and the predicted values [39]. The loss function employed is given by Eq. 1:

$$\ell(\theta) = - \sum_{h=1}^{H+1} \sum_{i=1}^{|\mathcal{V}_x|} 1_{\{z_{C+h+1}=i\}} \log p_{\theta}(z_{C+h+1} = i \mid z_{1:C+h}) \quad (1)$$

where H represents the forecast horizon, $z_{1:C+h}$ denotes the tokenized time-series, and $p_{\theta}(z_{C+h+1} = i \mid z_{1:C+h})$ is the model's predicted distribution [39].

Algorithm 1 shows the pseudocode for AmazonAICloud framework. The variable and equations in the Algorithm 1 are explained below:

- $X = \{x_1, x_2, \dots, x_n\}$: The original time-series data, where n is the total number of data points.
- C : The context length of the initial data points used for mean scaling.
- $\mu_a = \frac{1}{C} \sum_{i=1}^C |x_i|$: The mean of absolute values of the first C data points for normalising the scale of the data.
- m and s : Shift and scale factors (often $m = 0, s = 1$) for mean scaling.
- $X' = \frac{X-m}{\mu_a}$: The mean-scaled version of X .

- B : A binning parameter for uniform quantisation.
- $quant(x') = \text{round}\left(\frac{x' \times B}{B}\right)$: The uniform binning function that maps continuous values to discrete bins.
- $Z = \{z_1, z_2, \dots, z_n\}$: The sequence of tokens after quantisation.
- $L(\theta)$: The cross-entropy loss function, with θ denoting the model parameters.
- $p_{x,\theta}(z_i | z_1 : C + h)$: The predicted distribution of the token z_i given previous tokens up to index $C + h$.
- $z_{C+h+1} \sim p_{x,\theta}(\dots)$: Autoregressive sampling for generating future tokens.
- $inv_quant(z)$: The inverse quantisation function, converting tokens back to continuous values.
- $x_{\text{real}} = (x'' \times \mu_a) + m$: Maps scaled values back to the original numerical range.

Algorithm 1 The Pseudocode of AmazonAICloud Framework

(Step 1) Data Tokenization

- 1: $X = \{x_1, x_2, \dots, x_n\}$
- 2: $\mu_a = \frac{1}{C} \sum |x_i|, i = 1 \dots C$
- 3: $X' = \frac{X - m}{\mu_a}, m = 0 \text{ and } s = 1$
- 4: $\forall x' \in X', quant(x') = \text{round}\left(\frac{x' \times B}{B}\right)$
- 5: $Z = \{z_1, z_2, \dots, z_n\}$
- 6: PAD_token, $Z_{pad} = \{z_1, \dots, z_n, \text{PAD}, \dots, \text{PAD}\}$
- 7: EOS_token, $Z_{eos} = \{z_1, \dots, z_n, \text{EOS}\}$

(Step 2) Training the Model

- 8: $L(\theta) = - \sum \sum q(z_i) \log(p_{x,\theta}(z_i | z_1 : C + h))$

(Step 3) Making Predictions

- 9: $L(\theta) = - \sum \sum q(z_i) \log(p_{x,\theta}(z_i | z_1 : C + h))$
 - 10: $z_{C+h+1} \sim p_{x,\theta}(z_{C+h+1} | z_1 : C + h)$
 - 11: $x'' = inv_quant(z)$
 - 12: $x_{\text{real}} = (x'' \times \mu_a) + m$
 - 13: **End**
-

The steps of Algorithm 1 align directly within the framework's operations:

1. *Data tokenization*: The platform's historical workload data such as CPU usage is collected from the historical workload trace. The data is normalised and tokenized to prepare for inputting into the Chronos model. For example, historical spikes in CPU usage during past promotions, are collected and transformed into the format the model can process.
2. *Training the model*: Using the tokenized data, the model is trained to identify patterns in workload behaviour. For instance, the model learns that suddenly increasing requests (caused by viral events in this case) can continuously result

in steep increases in server usage over short periods. These patterns enable the model to predict demand spikes more accurately in future scenarios.

3. *Making predictions:* During real-time operations, the cloud resource manager sends the current resource usage metrics from physical servers (e.g., CPU and network usage). Based on these metrics, the trained Chronos model predicts upcoming workload behaviour. For example, the model might predict a 50% increase in resource demand over the next hour in the promotional event.

The cloud resource manager uses this prediction to allocate resources dynamically. If a spike is incoming, it provisions additional containers or VMs to meet the computing demand. By contrast, if the model predicts a decline in demand, it scales down the resources to reduce costs. This adaptive process ensures the platform maintains performance while optimising resource usage. The integration of the framework ensures that the shopping platform can handle dynamic and unpredictable workloads effectively.

3.2.4 Time complexity of the proposed framework

The overall time complexity of the proposed framework can be divided into three parts:

1. *Data tokenization:* Processing each data point takes constant time, leading to a complexity of $O(n)$, where n represents total number of data points.
2. *Model training:* Model processes the dataset in batches of size B . Each batch requires $O(H \times d)$ time for the forward and backward passes, where H represents the number of hidden states, and d is the model dimension. Then the total cost for training is $O\left(\frac{n}{B} \times H \times d\right)$.
3. *Making predictions:* Generating predictions for one time step involves $O(H \times d)$. If H represents both the hidden dimension and the number of time steps in the prediction phase, then the total inference complexity is $O(H \times d)$.

By summing these parts, the overall time complexity is given in Eq. 2:

$$O(n) + O\left(\frac{n}{B} \times H \times d\right) + O(H \times d) \quad (2)$$

4 Performance evaluation

This section discusses the evaluation metrics, the experimental setup, baselines, and the results.

4.1 Experimental setup

The proposed framework and baseline models were deployed in AWS SageMaker. AWS SageMaker is a cloud-based platform that facilitates the development and execution of ML models using Jupyter Notebook, it is highly scalable and provides on-demand access to computational resources such as CPUs and GPUs [49]. For the evaluation experiments, a CPU-based instance of ml.t3.large was configured to create the AWS SageMaker Jupyter Notebook. The instance includes a pre-configured environment with commonly used ML packages, enabling streamlined and convenient development of ML models. Further, details of testbed are given in our previous work [43].

4.2 Evaluation metrics

To assess models' predictive performance, we employ the evaluation metric including MAE, MSE, and MAPE. These metrics were tested on actual CPU usage and forecasted CPU usage to provide a comprehensive evaluation. MAE defines the average absolute error difference between original values and predicted outcomes, offering insight into the average deviation from the actual values [50]. MSE measures the average squared difference between original and predicted values, which is particularly sensitive to large errors, and penalises models for substantial deviations. A lower MSE generally indicates more accurate forecasting [51]. MAPE evaluates the absolute percentage error, which is crucial for understanding the average prediction error in relative terms, allowing for a detailed interpretation of model performance [52]. Different scoring functions highlight different aspects of prediction performance and can be associated with distinct statistical impact [53]. Specifically, MAE is consistent for the median function, MSE for the mean, and MAPE is commonly for its interpretability in percentage terms. Combining these evaluation metrics can gain complementary insights into the accuracy of model prediction, ensuring a thorough assessment of model reliability and consistency.

4.3 Baseline models

We evaluate the performance of the Amazon Chronos model within the AmazonAICloud framework for cloud resource management, comparing it against three baseline models: ARIMA [34], LSTM [40], and DeepAR [43].

4.3.1 ARIMA

The ARIMA model integrate autoregression and moving average models [54]. While it is widely used for various time series forecasting tasks, ARIMA performs best when the data is stationary, meaning its statistical properties remain constant over time [55]. However, resource usage patterns in cloud environments

are often non-stationary and highly dynamic, the ARIMA model might be less suitable for accurately predicting resource usage in such settings.

4.3.2 LSTM

LSTM is a type of Recurrent Neural Network (RNN), widely used in sequential data processing [37, 56]. Although the LSTM model works well with non-linear and non-stationary time series data [57], it is computationally intensive and more complex due to the large number of trainable parameters, which requires sophisticated design and data feature engineering to achieve high performance.

4.3.3 DeepAR

The DeepAR model developed by Salinas et al. [58] is an autoregressive RNN specialised in one-dimensional time series prediction. Unlike the ARIMA and LSTM models, DeepAR has the capability to predict multiple future values. The DeepAR model handles both stationary and non-stationary data and can capture complex behaviours such as trend and seasonality of the data [43].

4.4 Experiment results

The model was configured to forecast 30 samples of CPU usage for the next 30 min. Figure 5 shows the performance of the ARIMA model. It can be seen that the model struggles with complex time series. The performance of the LSTM model is shown in Fig. 6 and suggests that the model struggles with temporal data, making it a less optimal model for predicting the resource needed over time.

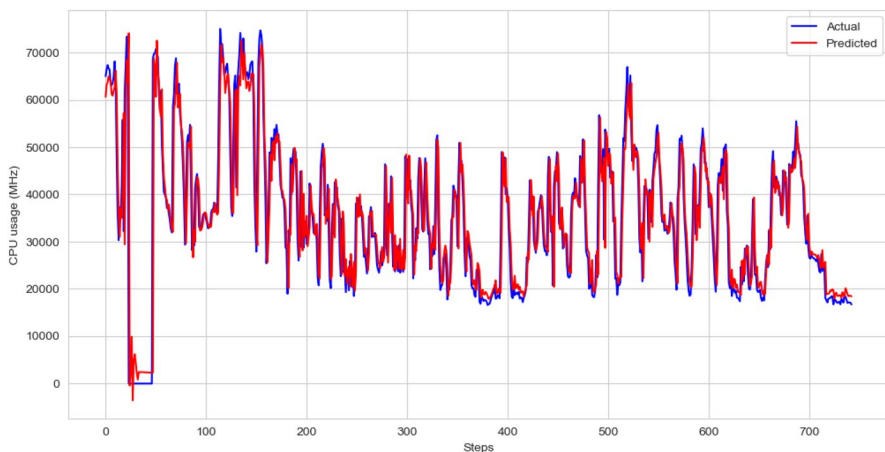


Fig. 5 ARIMA model: actual vs predicted CPU usage (MHz)

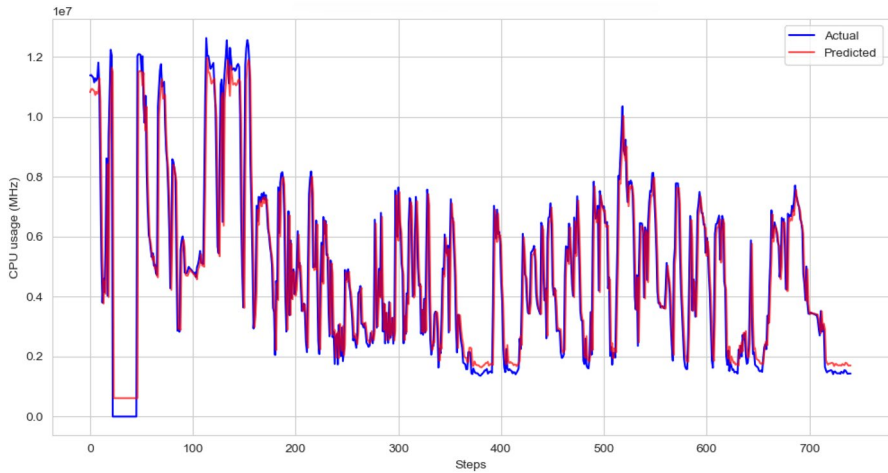


Fig. 6 LSTM model: actual vs predicted CPU usage (MHz)

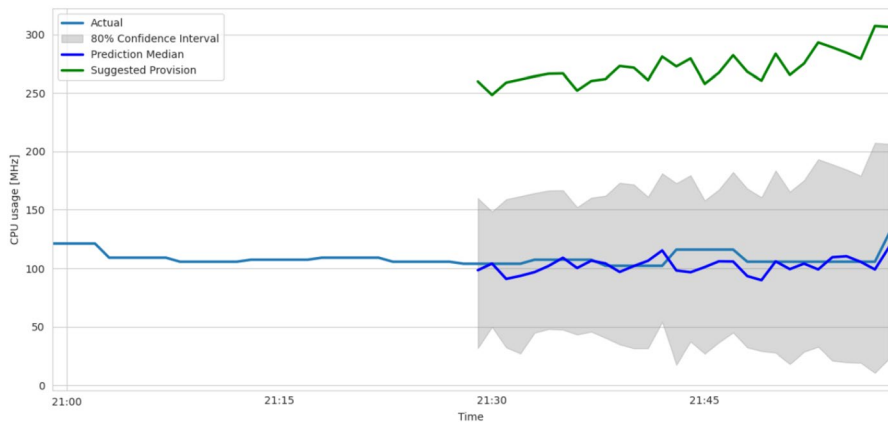


Fig. 7 DeepAR model: actual vs predicted CPU usage (MHz)

The performance of the DeepAR model is shown in Fig. 7. The performance of the Chronos model, as seen in Fig. 8, indicates that the model can provide reliable predictions.

4.4.1 Performance comparison with baseline models

Table 3 presents a comparison of the Chronos model with the baseline models in terms of performance metrics including MAE, MSE, and MAPE. Figure 9 provides a visual comparison among the models in terms of MAE. Figure 10 shows a comparison in terms of MSE, and Fig. 11 in terms of MAPE.

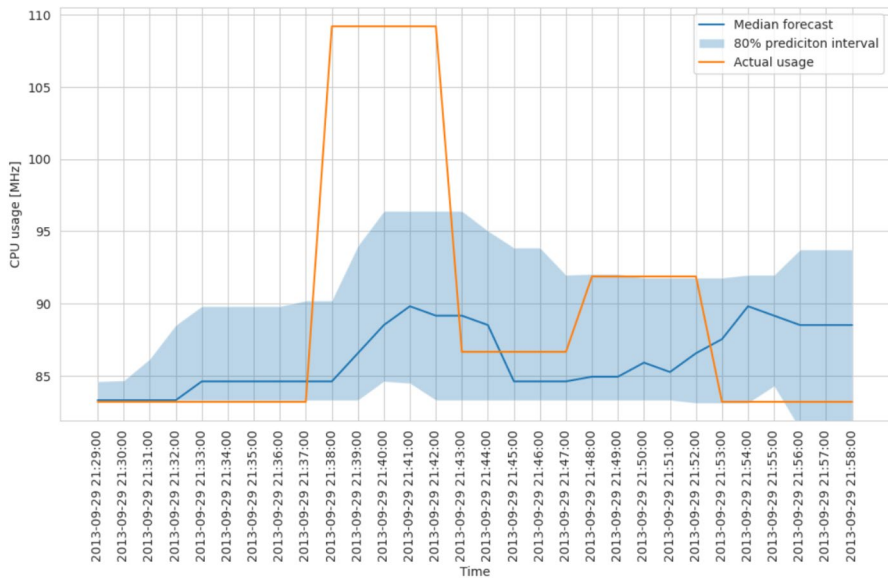


Fig. 8 Chronos model: actual vs predicted CPU usage (MHz)

Table 3 Performance comparison in terms of evaluation metrics and time

Work	AI Model	Evaluation metrics			Time	
		MAE	MSE	MAPE	Training	Inference
Calheiros et al. [34]	ARIMA	13.825	511.459	19.616	0.03	0.012
Tang et al. [40]	LSTM	8.217	1728.004	24.550	40.8	0.6
Velu et al. [43]	DeepAR	15.852	230.884	7.550	562	1.7
AmazonAICloud	Amazon Chronos	6.332	91.390	6.450	324	1.27

Fig. 9 Comparison of chronos model performance with base-lines in terms of MAE

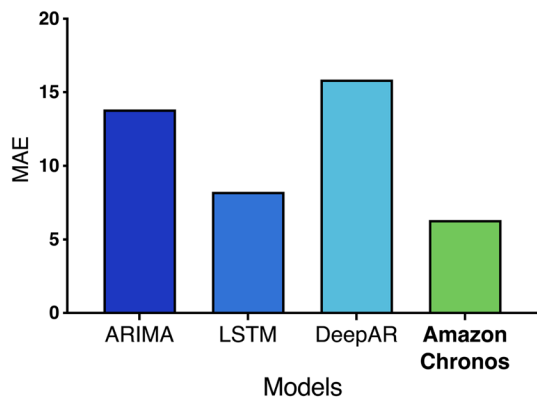


Fig. 10 Comparison of chronos model performance with base-lines in terms of MSE

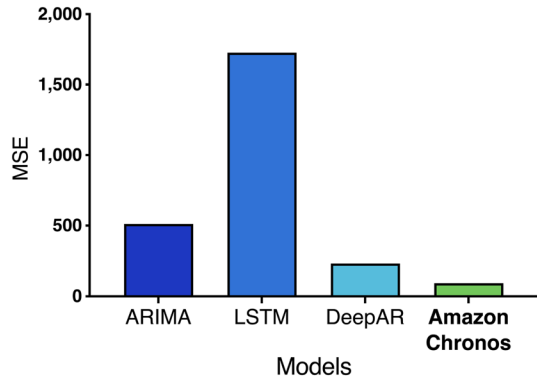
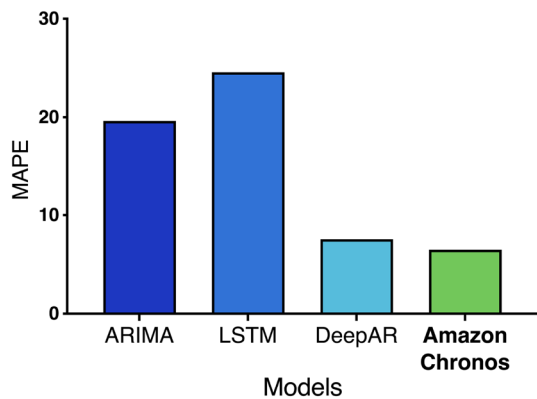


Fig. 11 Comparison of chronos model performance with base-lines in terms of MAPE



The ARIMA model shows a high MAE of 13.825 and a significantly high MSE of 511.459. This indicates a significant difference between the model's predicted values and the actual values. The MAPE of 19.616 suggests that although the model's accuracy is relatively poor, it can still be an acceptable approach for cloud resource allocation. Although the LSTM model has demonstrated a lower MAE of 8.217, compared to the ARIMA model, it has exhibited the highest MSE of 1728.004 and highest MAPE of 24.55, suggesting large deviations from true values. The large MAPE suggests that LSTM is a poor approach and could lead to unfavourable resource allocation decisions, resulting in poor user experience.

The DeepAR model, when compared to other baseline models, has lower MSE and MAPE of 230.884 and 7.55 respectively. The comparatively lower metrics suggest that the model is consistent and provides accurate predictions. However, the large MAE value of 15.852 suggests larger deviations from the original values, which can be improved with some hyperparameter tuning. On the other hand, the proposed Chronos model produced better results across all the metrics. The lower MAE of 6.332 indicates that the model does not portray much deviation from the true values. The significantly lower MSE of 91.39 suggests that the Chronos model predicts with a lower error and is well within the actual values.

Finally, the MAPE of 6.45 suggests that the model's results are quite accurate and consistent with the actual values. These results indicate that the Chronos model is a viable alternative that may be employed in cloud resource management systems, having the capability of handling complex and non-stationary data frequently seen in cloud computing environments.

4.4.2 Analysis of model training and prediction times

Table 3 presents a comparison of training and inference times for the Chronos model against the baseline models including ARIMA, LSTM, and DeepAR. The results demonstrate that ARIMA is the fastest model in both training and inference, making it computationally efficient for simpler tasks, while ARIMA model's performance is often limited in handling complex, non-stationary datasets, as discussed earlier in this study. The LSTM model, while significantly faster than DeepAR and Chronos, demonstrates higher error rates as mentioned in Table 3. DeepAR model exhibits the highest training time of 562 s, making it the most computationally intensive model among all the comparisons. In contrast, Chronos model achieved a training time of 324 s, achieving a 42.35% reduction in training time compared to DeepAR model. In terms of the inference times, ARIMA and LSTM models were able to generate predictions more quickly, whereas DeepAR and Chronos shows a relatively higher inference times.

5 Conclusions and future work

In this paper, we proposed a cloud resource management architecture called AmazonAICloud to optimise cloud workload prediction. The AmazonAICloud framework integrates existing cloud architectures with Amazon Chronos time-series prediction model, which proactively allocates resources based on predicted usage, aiming for more sustainable cloud resource provisioning. Our experiments demonstrate that Chronos consistently outperforms baseline models such as ARIMA, LSTM, and DeepAR in terms of different evaluation metrics including MAE, MSE and MAPE and time complexity, achieving higher accuracy in forecasting resource demands. The improvement in prediction accuracy not only contributes to reduced operational cloud services overhead, but also supports the goal of environmentally sustainable cloud solutions by minimising resource wastage.

5.1 Future directions

Even though there are evident advantages brought by Amazon Chronos, AmazonAICloud framework still has space for further improvement. The findings of this study open numerous possibilities for future exploration. This framework is built upon architecture presented in previous works [39, 43] and does not include other models. Incorporating other hybrid ML models could further enhance its efficiency under different scenarios.

Chronos model can be incorporated into existing cloud resource management architectures and tested using real-time cloud performance metrics within current resource allocation systems. While the Chronos tiny model used in this study is lightweight and computationally efficient, future work could explore the potential of higher capacity versions (such as base and large) to handle more complex workloads.

Generative AI models could also be employed to simulate diverse resource usage patterns, allowing for the testing of various allocation strategies [10, 59]. This way could ensure the optimal performance while improving cost and energy efficiency.

Additionally, future work could also focus on investigating energy consumption and carbon emission optimised by the proposed framework. Exploring explainable AI could also enhance the transparency and trustworthiness of the model's decisions, ensuring compliance with Service Level Agreement (SLA) requirements and improving its practical applicability.

6 Software availability

This work has been made available as open-source. All code, datasets, and result reproducibility scripts are publicly accessible on GitHub: <https://github.com/HTXW/AmazonAICloud>

Author contributions Han Wang: Conceptualisation: Lead; Data curation: Lead; Formal analysis: Lead; Investigation: Lead; Methodology: Lead; Software: Lead; Validation: Lead; Writing—original draft: Lead; Writing—review and editing: Lead. Kevin Joji Mathews: Conceptualisation: Lead; Data curation: Supporting; Formal analysis: Supporting; Investigation: Supporting; Methodology: Supporting; Software: Supporting; Validation: Supporting; Writing – original draft: Supporting. Muhammed Golec: Writing—review and editing: Supporting. Sukhpal Singh Gill: Supervision: Lead; Writing—original draft: Lead; Writing—review and editing: Lead. Steve Uhlig: Supervision: Supporting; Writing—original draft: Lead; Writing—review and editing: Supporting

Funding Not applicable.

Data availability Not applicable.

Materials availability Not applicable.

Code availability Not applicable.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

References

- Gammelgaard B, Nowicka K (2024) Next generation supply chain management: the impact of cloud computing. *J Enterpr Inf Manag* 37(4):1140–1160. <https://doi.org/10.1108/JEIM-09-2022-0317>
- Botta A, De Donato W, Persico V, Pescapé A (2016) Integration of cloud computing and internet of things: a survey. *Futur Gener Comput Syst* 56:684–700. <https://doi.org/10.1016/j.future.2015.09.021>
- Duan S, Wang D, Ren J, Lyu F, Zhang Y, Wu H, Shen X (2023) Distributed artificial intelligence empowered by end-edge-cloud computing: a survey. *IEEE Commun Surveys Tutor* 25(1):591–624. <https://doi.org/10.1109/COMST.2022.3218527>
- Varghese B, Buyya R (2018) Next generation cloud computing: new trends and research directions. *Futur Gener Comput Syst* 79:849–861. <https://doi.org/10.1016/j.future.2017.09.020>
- Salimian M, Ghobaei-Arani M, Shahidinejad A (2022) An evolutionary multi-objective optimization technique to deploy the IoT services in fog-enabled networks: an autonomous approach. *Appl Artif Intell* 36(1):2008149. <https://doi.org/10.1080/08839514.2021.2008149>
- Makridakis S (2017) The forthcoming artificial intelligence (AI) revolution: its impact on society and firms. *Futures* 90:46–60. <https://doi.org/10.1016/j.futures.2017.03.006>
- Yao Y, Duan J, Xu K, Cai Y, Sun Z, Zhang Y (2024) A survey on large language model (LLM) security and privacy: the good, the bad, and the ugly. *High-Confid Comput* 4(2):100211. <https://doi.org/10.1016/j.hcc.2024.100211>
- Rajpurkar P, Chen E, Banerjee O, Topol EJ (2022) AI in health and medicine. *Nat Med* 28(1):31–38. <https://doi.org/10.1038/s41591-021-01614-0>. (Accessed 2025-01-08)
- ...Gill SS, Wu H, Patros P, Ottaviani C, Arora P, Pujol VC, Haunschild D, Parlikad AK, Cetinkaya O, Lutfiyya H, Stankovski V, Li R, Ding Y, Qadir J, Abraham A, Ghosh SK, Song HH, Sakellariou R, Rana O, Rodrigues JJPC, Kanhere SS, Dustdar S, Uhlig S, Ramamohanarao K, Buyya R (2024) Modern computing: vision and challenges. *Telemat Inf Rep* 13:100116. <https://doi.org/10.1016/j.teler.2024.100116>
- Gill SS, Xu M, Ottaviani C, Patros P, Bahsoon R, Shaghagh A, Golec M, Stankovski V, Wu H, Abraham A et al (2022) Ai for next generation computing: emerging trends and future directions. *Int Things* 19:100514. <https://doi.org/10.1016/j.iot.2022.100514>
- Yang J, Xiao W, Jiang C, Hossain MS, Muhammad G, Amin SU (2019) Ai-powered green cloud and data center. *IEEE Access* 7:4195–4203. <https://doi.org/10.1109/ACCESS.2018.2888976>
- Ghorbian M, Ghobaei-Arani M, Esmaeili L (2024) A survey on the scheduling mechanisms in serverless computing: a taxonomy, challenges, and trends. *Clust Comput* 27(5):5571–5610. <https://doi.org/10.1007/s10586-023-04264-8>
- Zhou Q, Xu M, Gill SS, Gao C, Tian W, Xu C, Buyya R (2020) Energy efficient algorithms based on vm consolidation for cloud computing: comparisons and evaluations. In: 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID), pp. 489–498. doi: <https://doi.org/10.1109/CCGrid49817.2020.00-44>. IEEE
- Welsh T, Benkhelifa E (2020) On resilience in cloud computing: a survey of techniques across the cloud domain. *ACM Comput Surv* 53(3) <https://doi.org/10.1145/3388922>
- Jazayeri F, Shahidinejad A, Ghobaei-Arani M (2021) A latency-aware and energy-efficient computation offloading in mobile fog computing: a hidden Markov model-based approach. *J Supercomput* 77(5):4887–4916. <https://doi.org/10.1007/s11227-020-03476-8>
- Singh S, Chana I (2016) Cloud resource provisioning: survey, status and future research directions. *Knowl Inf Syst* 49(3):1005–1069. <https://doi.org/10.1007/s10115-016-0922-3>
- Yousafzai A, Gani A, Noor RM, Sookhak M, Talebian H, Shiraz M, Khan MK (2017) Cloud resource allocation schemes: review, taxonomy, and opportunities. *Knowl Inf Syst* 50(2):347–381. <https://doi.org/10.1007/s10115-016-0951-y>
- Mampage A, Karunasekera S, Buyya R (2022) A holistic view on resource management in serverless computing environments: taxonomy and future directions. *ACM Comput Surv* 54(11s) <https://doi.org/10.1145/3510412>
- Khan T, Tian W, Zhou G, Ilager S, Gong M, Buyya R (2022) Machine learning (ML)-centric resource management in cloud computing: a review and future directions. *J Netw Comput Appl* 204:103405. <https://doi.org/10.1016/j.jnca.2022.103405>

20. Chan KY, Abu-Salih B, Qaddoura R, Al-Zoubi AM, Palade V, Pham D-S, Ser JD, Muhammad K (2023) Deep neural networks in the cloud: review, applications, challenges and research directions. *Neurocomputing* 545:126327. <https://doi.org/10.1016/j.neucom.2023.126327>
21. Benlian A, Kettinger WJ, Sunyaev A, Winkler TJ (2018) GUEST EDITORS: Special section: the transformative value of cloud computing: a decoupling, platformization, and recombination theoretical framework. *J Manag Inf Syst* 35(3):719–739. <https://doi.org/10.1080/07421222.2018.1481634>
22. Gill SS, Buyya R (2019) A taxonomy and future directions for sustainable cloud computing: 360 degree view. *ACM Comput Surv* 51(5):1–33. <https://doi.org/10.1145/3241038>
23. Gaur L, Afaq A, Arora GK, Khan N (2023) Artificial intelligence for carbon emissions using system of systems theory. *Eco Inform* 76:102165. <https://doi.org/10.1016/j.ecoinf.2023.102165>
24. Shaji George A, Hovan George A, Martin A (2023) The environmental impact of AI: a case study of water consumption by chat GPT. *Partners Univ Int Innov J* 2(1):97–104. <https://doi.org/10.5281/zenodo.7855594>
25. Wang Q, Li Y, Li R (2024) Ecological footprints, carbon emissions, and energy transitions: the impact of artificial intelligence (AI). *Human Soc Sci Commun* 11(1) <https://doi.org/10.1057/s41599-024-03520-5>
26. International Energy Agency (2024) Electricity 2024. Technical report, International Energy Agency, Paris. <https://www.iea.org/reports/electricity-2024>
27. Bermejo B, Juiz C (2023) Improving cloud/edge sustainability through artificial intelligence: a systematic review. *J Par Distrib Comput* 176:41–54. <https://doi.org/10.1016/j.jpdc.2023.02.006>
28. Aghazadeh R, Shahidinejad A, Ghobaei-Arani M (2023) Proactive content caching in edge computing environment: a review. *Softw: Pract Exp* 53(3):811–855. <https://doi.org/10.1002/spe.3033>
29. Tari M, Ghobaei-Arani M, Pouramini J, Ghorbian M (2024) Auto-scaling mechanisms in serverless computing: a comprehensive review. *Comput Sci Rev* 53:100650. <https://doi.org/10.1016/j.cosrev.2024.100650>
30. Dewangan BK, Agarwal A, Choudhury T, Pasricha A, Chandra Satapathy S (2021) Extensive review of cloud resource management techniques in industry 4.0: issue and challenges. *Softw: Pract Exp* 51(12):2373–2392. <https://doi.org/10.1002/spe.2810>
31. Jennings B, Stadler R (2015) Resource management in clouds: survey and research challenges. *J Netw Syst Manag* 23(3):567–619. <https://doi.org/10.1007/s10922-014-9307-7>
32. Shen H, Chen L (2018) Resource demand misalignment: an important factor to consider for reducing resource over-provisioning in cloud datacenters. *IEEE/ACM Trans Netw* 26(3):1207–1221. <https://doi.org/10.1109/TNET.2018.2823642>
33. Mahida A (2022) Comprehensive review on optimizing resource allocation in cloud computing for cost efficiency. *J Artif Intell Cloud Comput*, 1–4 [https://doi.org/10.47363/JAICC/2022\(1\)232](https://doi.org/10.47363/JAICC/2022(1)232)
34. Calheiros RN, Masoumi E, Ranjan R, Buyya R (2014) Workload prediction using arima model and its impact on cloud applications' qos. *IEEE Trans Cloud Comput* 3(4):449–458. <https://doi.org/10.1109/TCC.2014.2350475>
35. Zhang Y, Liu B, Gong Y, Huang J, Xu J, Wan W (2024) Application of machine learning optimization in cloud computing resource scheduling and management. *CIBDA '24*, pp. 171–175. Association for Computing Machinery, New York, NY, USA. doi: <https://doi.org/10.1145/3671151.3671183>
36. Lim B, Zohren S (2021) Time-series forecasting with deep learning: a survey. *Philos Trans Royal Soc A: Math, Phys Eng Sci* 379(2194):20200209. <https://doi.org/10.1098/rsta.2020.0209>
37. Christofidi G, Papaioannou K, Doudali TD (2023) Is machine learning necessary for cloud resource usage forecasting? In: *Proceedings of the 2023 ACM Symposium on Cloud Computing*. SoCC '23, pp. 544–554. Association for Computing Machinery, New York, NY, USA. doi: <https://doi.org/10.1145/3620678.3624790>
38. Wang S, Li C, Lim A (2021) Why Are the ARIMA and SARIMA not Sufficient. <https://arxiv.org/abs/1904.07632>
39. Ansari AF, Stella L, Turkmen C, Zhang X, Mercado P, Shen H, Shchur O, Rangapuram SS, Arango SP, Kapoor S, Zschiegner J, Maddix DC, Wang H, Mahoney MW, Torkkola K, Wilson AG, Bohlke-Schneider M, Wang Y (2024) Chronos: Learning the Language of Time Series. <https://arxiv.org/abs/2403.07815>
40. Tang X, Liu Q, Dong Y, Han J, Zhang Z (2018) Fisher: An efficient container load prediction model with deep neural network in clouds. In: *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*, pp. 199–206. doi: <https://doi.org/10.1109/BDCloud.2018.00041>. IEEE

41. Yan M, Liang X, Lu Z, Wu J, Zhang W (2021) Hansel: adaptive horizontal scaling of microservices using BI-LSTM. *Appl Soft Comput* 105:107216. <https://doi.org/10.1016/j.asoc.2021.107216>
42. Ali T, Khan HU, Alarfaj FK, AlReshoodi M (2024) Hybrid deep learning and evolutionary algorithms for accurate cloud workload prediction. *Computing* 106(12):3905–3944. <https://doi.org/10.1007/s00607-024-01340-8>
43. Velu S, Gill SS, Murugesan SS, Wu H, Li X (2024) Cloudaibus: a testbed for AI based cloud computing environments. *Cluster Comput*. <https://doi.org/10.1007/s10586-024-04562-9>
44. Bitbrains: GWA-T-12 Bitbrains. <http://gwa.ewi.tudelft.nl/datasets/gwa-t-12-bitbrains>
45. Shen S, Van Beek V, Iosup A (2015) Statistical characterization of business-critical workloads hosted in cloud datacenters. In: 2015 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, IEEE. pp 465–474. <https://doi.org/10.1109/CCGrid.2015.60>
46. Kwiatkowski D, Phillips PCB, Schmidt P, Shin Y (1992) Testing the null hypothesis of stationarity against the alternative of a unit root: How sure are we that economic time series have a unit root? *J Economet* 54(1):159–178. [https://doi.org/10.1016/0304-4076\(92\)90104-Y](https://doi.org/10.1016/0304-4076(92)90104-Y)
47. Liu P, Wu B, Hu Y, Li N, Dai T, Bao J, Xia St (2024) TimeBridge: Non-Stationarity Matters for Long-term Time Series Forecasting (2024). <https://arxiv.org/abs/2410.04442>
48. Gunawi HS, Hao M, Suminto RO, Laksono A, Satria AD, Adityatama J, Eliazar KJ (2016) Why does the cloud stop computing? lessons from hundreds of service outages. In: Proceedings of the Seventh ACM Symposium on Cloud Computing. SoCC '16, pp. 1–16. Association for Computing Machinery, New York, NY, USA. doi: <https://doi.org/10.1145/2987550.2987583>
49. Amazon: Amazon SageMaker. <https://aws.amazon.com/sagemaker/>. [Accessed 01-01-2025] (2025)
50. Cort J (2005) Willmott, Kenji Matsuura: Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate Res* 30(1):79–82. <https://doi.org/10.3354/cr030079>
51. Hodson TO (2022) Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. *Geosci Model Dev* 15(14):5481–5487. <https://doi.org/10.5194/gmd-15-5481-2022>
52. De Myttenaere A, Golden B, Le Grand B, Rossi F (2016) Mean absolute percentage error for regression models. *Neurocomputing* 192:38–48. <https://doi.org/10.1016/j.neucom.2015.12.114>
53. Gneiting T (2011) Making and evaluating point forecasts. *J Am Stat Assoc* 106(494):746–762. <https://doi.org/10.1198/jasa.2011.r10138>
54. Gupta A, Kumar A (2020) Mid term daily load forecasting using arima, wavelet-arima and machine learning. In: 2020 IEEE International Conference on Environment and Electrical Engineering and 2020 IEEE Industrial and Commercial Power Systems Europe (EEEIC/I & CPS Europe), pp. 1–5. doi: <https://doi.org/10.1109/IEEEIC/ICPSEurope49358.2020.9160563>. IEEE
55. Cheng C, Sa-Ngasongsong A, Beyca O, Le T, Yang H, Kong ZJ, Bukkapatnam STS (2015) Time series forecasting for nonlinear and non-stationary processes: a review and comparative study. *IIE Trans* 47(10):1053–1071. <https://doi.org/10.1080/0740817X.2014.999180>
56. Sherstinsky A (2020) Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. *Physica D* 404:132306. <https://doi.org/10.1016/j.physd.2019.132306>
57. Preeti, Bala R, Singh RP (2019) Financial and non-stationary time series forecasting using lstm recurrent neural network for short and long horizon. In: 2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT), pp. 1–7. doi: <https://doi.org/10.1109/ICCCNT45670.2019.8944624>
58. Salinas D, Flunkert V, Gasthaus J, Januschowski T (2020) Deepar: Probabilistic forecasting with autoregressive recurrent networks. *Int J Forecast* 36(3):1181–1191. <https://doi.org/10.1016/j.ijforecast.2019.07.001>
59. Mendes AH, Rosa MJ, Marotta MA, Araujo A, Melo AC, Ralha CG (2024) Mas-cloud+: A novel multi-agent architecture with reasoning models for resource management in multiple providers. *Futur Gener Comput Syst* 154:16–34. <https://doi.org/10.1016/j.future.2023.12.022>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Han Wang¹ · Kevin Joji Mathews¹ · Muhammed Golec¹ · Sukhpal Singh Gill¹ · Steve Uhlig¹

✉ Sukhpal Singh Gill
s.s.gill@qmul.ac.uk

Han Wang
hanwang@qmul.ac.uk

Kevin Joji Mathews
k.jojimathews@se23.qmul.ac.uk

Muhammed Golec
m.golec@qmul.ac.uk

Steve Uhlig
steve.uhlig@qmul.ac.uk

¹ School of Electronic Engineering and Computer Science, Queen Mary University of London, Mile End Road, London E1 4NS, UK